

访问控制漏洞报告（Lab3：用户角色由请求参数控制）

1. 漏洞名称

访问控制失效（Broken Access Control） — 客户端 Cookie 权限篡改

2. 漏洞等级

高危（High）

3. 漏洞描述

该网站的管理员面板路径为 /admin，但后端身份校验机制存在设计缺陷。服务器**完全信任客户端浏览器提交的 Cookie 参数 Admin**。攻击者在登录普通账号后，只需通过浏览器开发者工具将 Cookie 中的 Admin=false 修改为 Admin=true，即可绕过服务端权限校验，直接以管理员身份访问后台并删除任意用户。

4. 漏洞位置

攻击向量：Cookie: Admin=...

未授权访问路径：GET /admin

5. 漏洞复现过程

第一步：普通用户登录与访问受限

使用提供的测试账号 wiener:peter 进行登录。登录成功后，尝试访问 /admin 管理面板，页面提示无权限。

图 1：普通用户无法直接访问后台



说明：普通用户访问 /admin 被拦截，证明存在基础的身份校验逻辑。

第二步：查看与管理当前身份标识

登录后，打开浏览器的开发者工具（F12），切换到“应用程序/存储 (Application)”标签页，查看当前的 Cookie。

图 2：查看本地存储的权限 Cookie

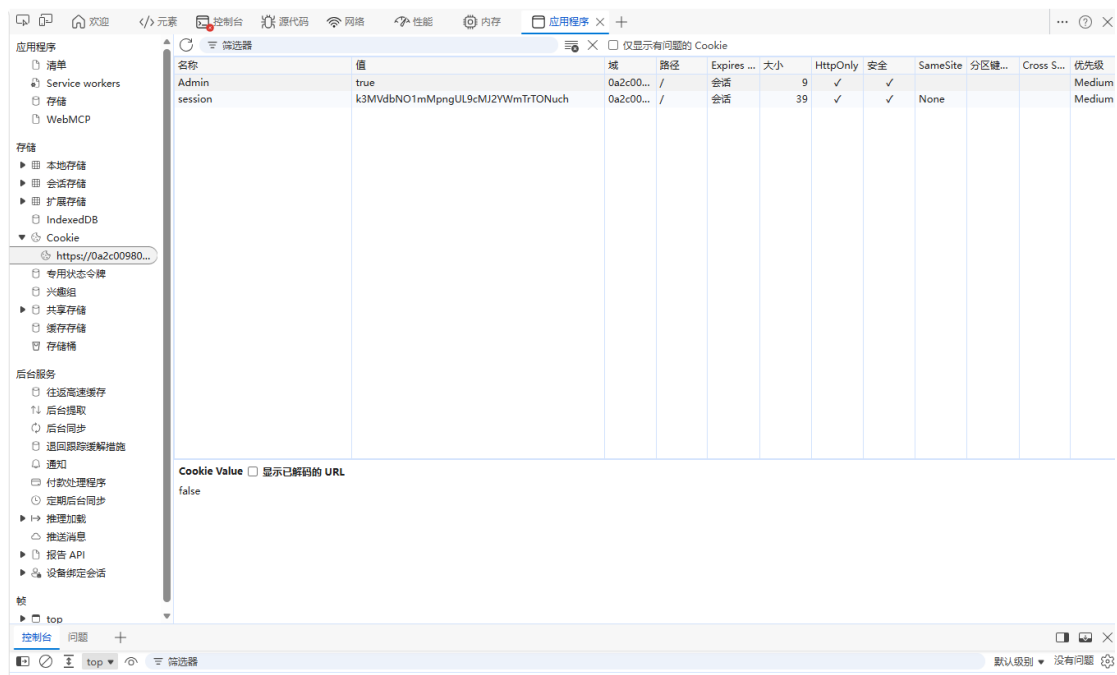


说明：发现当前登录状态下的权限标识 Admin 被保存在了客户端 Cookie 中，值为 false。

第三步：篡改请求参数，提权为管理员

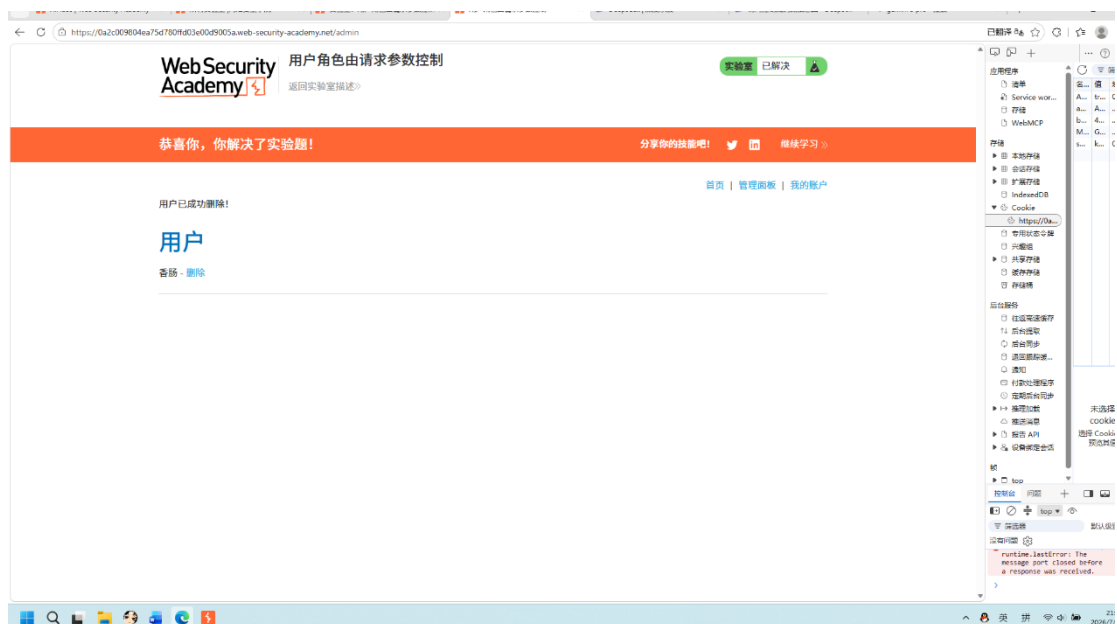
在开发者工具的 Cookie 面板中，双击 Admin 对应的“值”列，将 false 修改为 true。刷新页面或重新访问 /admin。

图 3：成功进入管理面板



第四步：执行删除操作

在管理面板中，点击用户 carlos（香肠）右侧的“删除”按钮。页面提示删除成功，完成实验。



6. 漏洞原理

该漏洞属于水平越权与功能级访问控制缺失的结合体。

1. 开发者试图在用户浏览器中保存状态，于是在登录响应中设置了 Admin=false。

2. **严重安全缺陷**：后端在处理请求时，直接读取了 Cookie 中的 Admin 值来判断用户身份，而没有去服务器端的 Session 存储中校验真正的用户角色。
3. 攻击者只要掌握了 Cookie 的修改方法，就可以完全控制自己的权限状态。

7. 漏洞影响

- **权限绕过**：任何拥有普通账号的攻击者，无需暴力破解，直接修改 1 个 Cookie 字符即可瞬间获取最高权限。
- **业务安全风险**：攻击者可接管后台、删除所有数据、修改系统配置。

8. 修复建议

- ① **绝不信任客户端数据**：绝对禁止将 isAdmin、role 等关键权限标识存放在 Cookie 或前端隐藏域中。权限标识必须**仅在服务端 Session 或 JWT 的 Payload 中存储**。
- ② **服务端强制校验**：每当用户访问敏感接口（如 /admin）时，后端必须从服务端存储（如 Redis/Session）中提取用户的真实角色进行比对，而不是看前端传过来的参数。
- ③ **使用 HttpOnly 和 Secure 标志**：对于 Session ID 等敏感 Cookie，必须设置 HttpOnly 防止 JS 读取，设置 Secure 防止传输劫持。

9. 工具

- **Google Chrome / Firefox**（开发者工具/应用程序面板修改 Cookie）
- **PortSwigger Web Security Academy**（靶场平台）